Ray Cluster Management Workshop

Progress Report — Workshop Delivered and Work Completed

Jahangir

19 June 2026

Abstract

This report summarises the Ray Cluster Management Workshop delivered on 16 June 2026 on a live eight-node GPU cluster (268 CPUs, 8 GPUs, 192 GB VRAM total). The session was structured as four progressive parts — Foundations, Deployment, Using the Cluster, and Hands-On Examples — and covered the complete workflow with briefing of bare-metal cluster setup and practical demonstration of matrix multiplications, distributed training of a Llama 3.1-8B model and multi-node LLM inference serving via vLLM and Ray. The report describes what was demonstrated, includes diagrams reproduced from the presentation slides, and records the key performance results obtained on the live cluster during the session.

Contents

1 Executive Summary

The workshop guided participants through deploying and operating a production Ray cluster, progressing from first-principles understanding of the Ray framework to hands-on distributed training of large language models and multi-node inference serving. All demonstrations ran on live hardware visible to the room.

Two complementary bodies of work were presented.

1. **A deployed cluster and training pipeline.** A physical eight-node cluster — five RTX 4500 Ada and three RTX 3090 GPUs, all connected over a 1 Gb Ethernet switch — was set up from scratch using the Manual CLI method and used to run distributed PyTorch training from ResNet-152 DDP through FSDP-sharded Llama 3.1-8B. All experiments were tracked in MLflow with metrics, parameters, and model checkpoints stored on NFS shared storage.

2. **A structured workshop curriculum.** The accompanying presentation (38 slides, four parts) explained Ray's architecture, the three deployment strategies, cluster usage patterns, and the vLLM multi-node serving pipeline, framed around live results from the cluster.

The headline performance result, demonstrated live, was a **9.2× speedup** on a 300-task matrix-multiplication benchmark: 156 s on a single machine versus 17 s on the eight-node cluster. The Llama 3.1-8B FSDP training job completed with status **SUCCEEDED** in **2 h 49 min** across all eight GPUs.

2 Cluster Infrastructure

2.1 Hardware

The physical cluster comprises eight heterogeneous GPU nodes on a shared 1 Gb LAN. Table 1 lists each node and its role.

Table 1. *Physical cluster nodes and assigned roles.*

Node	IP	GPU	VRAM	Role
pc3-4500	192.168.3.73	RTX 4500 Ada	24 GB	Head + Worker
pc1-4500	192.168.3.71	RTX 4500 Ada	24 GB	Worker 1
pc2-4500	192.168.3.72	RTX 4500 Ada	24 GB	Worker 2
pc4-4500	192.168.3.74	RTX 4500 Ada	24 GB	Worker 3
pc5-4500	192.168.3.75	RTX 4500 Ada	24 GB	Worker 4
pc6-3090	192.168.3.76	RTX 3090	24 GB	Worker 5
pc7-3090	192.168.3.77	RTX 3090	24 GB	Worker 6
pc8-3090	192.168.3.78	RTX 3090	24 GB	Worker 7
Total		8 GPUs	192 GB	268 CPUs

Figure 1 reproduces the cluster architecture diagram from the presentation, showing the services hosted on the head node and the seven task-execution workers.

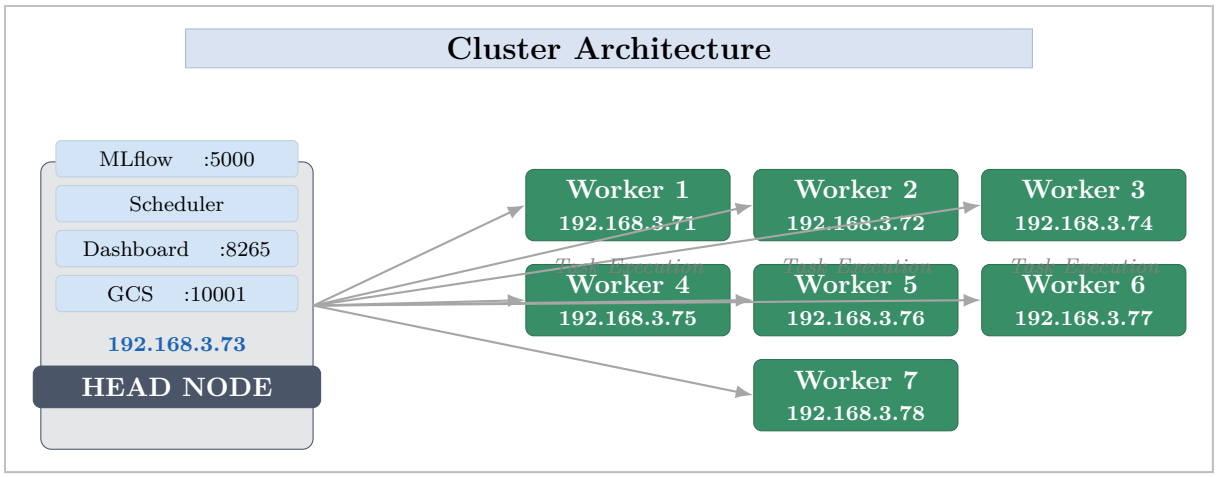


Figure 1. *Cluster architecture as presented: the head node (192.168.3.73) hosts the Global Control Store, Dashboard, Scheduler, and MLflow server; the seven worker nodes execute distributed tasks.*

2.2 Network Topology

All nodes connect through a single 1 Gb Ethernet switch. The network speed was verified on each node with `ethtool`, confirming `Speed: 1000Mb/s` and `Link detected: yes` on every interface. Figure 2 reproduces the topology diagram from the presentation.

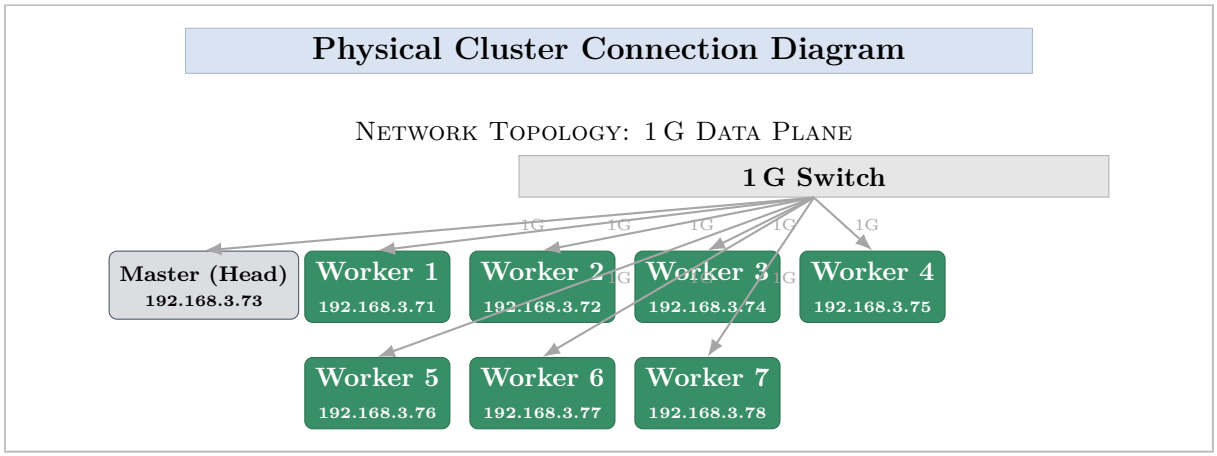


Figure 2. *Physical network topology: all eight nodes connect to a single 1 Gb Ethernet switch. Verified link speed: 1000 Mb/s on every node.*

3 Workshop Structure

The workshop was delivered as four sequential parts, each building on the previous. Table 2 shows the full outline.

Table 2. *Four-part workshop outline.*

Part	Title	Topics covered
1	Foundations	What is Ray?, Ecosystem, Physical Cluster, Network
2	Deployment	Methods, Installation, Dashboard, MLflow, NFS
3	Using the Cluster	Connection, Job submission, Matrix mult., Train, vLLM
4	Hands-On Examples	Ray Core, Ray Train, vLLM on 8 GPUs

4 Part 1 — Foundations

4.1 What is Ray?

Ray is an open-source distributed computing framework that scales Python and AI workloads from a single machine to a cluster of thousands of nodes. The core value proposition is that the programmer writes ordinary Python; Ray's scheduler, object store, and fault-tolerant runtime handle the distribution transparently. The same code runs on a laptop or a 1 000-node cluster, and native GPU resource tracking means no manual device placement across heterogeneous nodes.

4.2 The Ray Ecosystem

Figure 3 reproduces the ecosystem architecture from the presentation, showing the layered stack from compute infrastructure through Ray Core and Ray's high-level libraries, and the position of vLLM as an optional execution backend that sits on top of the same physical compute layer.

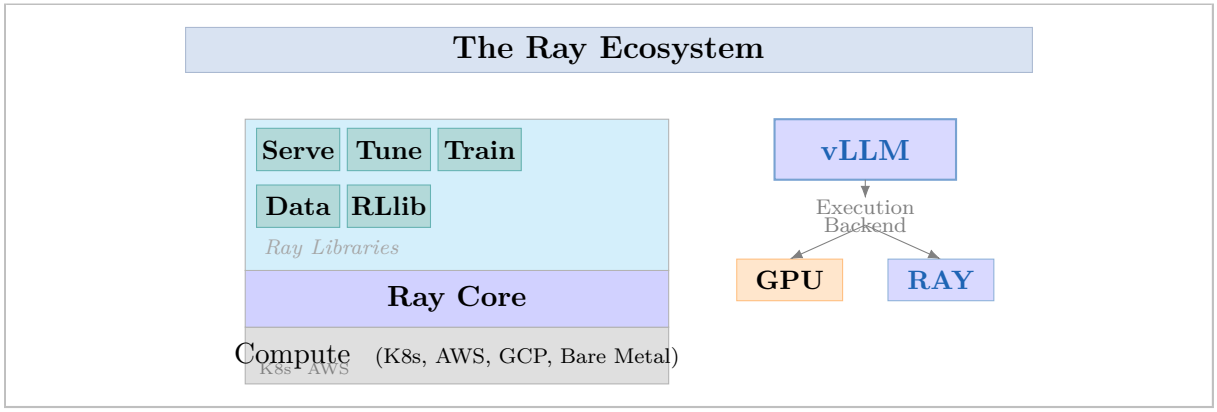


Figure 3. *The Ray ecosystem: compute infrastructure underpins Ray Core, which in turn supports six high-level libraries. vLLM uses Ray as its distributed executor backend, sitting on the same compute layer.*

Table 3 summarises each library's role as introduced in the presentation.

Table 3. *Ray library ecosystem.*

Library	Role
Ray Core	Low-level `@ray.remote` task and actor API — the foundation for all higher libraries
Ray Train	Distributed training for PyTorch, TensorFlow, XGBoost, HuggingFace; supports DDP and FSDP
Ray Tune	Hyperparameter optimisation with pluggable backends (Optuna, ASHA, PBT); thousands of parallel trials
Ray Serve	Online model serving with autoscaling; any Python function as an HTTP endpoint
Ray Data	Distributed data loading and preprocessing; chains natively into Train/Tune pipelines
Ray RLlib	Reinforcement learning at scale (PPO, SAC, etc.); automatic rollout worker distribution

5 Part 2 — Deployment

5.1 Deployment methods

Three deployment strategies were introduced and compared. The workshop focused entirely on the **Manual CLI** method, which is most transparent for learning and is appropriate for fixed bare-metal clusters.

Table 4. *Deployment method comparison as shown in the presentation.*

Method	Environment	Complexity	Auto-scale	Best For
Manual CLI	Bare metal / VMs	Low	No	Learning, fixed clusters
Docker Compose	Single host	Medium	Manual	Dev, demos, CI/CD
KubeRay	Kubernetes	High	Yes	Production, cloud-native

5.2 Installation and cluster startup

Ray was installed identically on every node using a Conda environment:

Listing 1. *Installation — same steps on all 8 nodes.*

```
conda create -n ray-env python=3.12 -y
conda activate ray-env
pip install ray[default]
```

The head node was started with dashboard exposed on all interfaces, then each worker joined by pointing at the head node's GCS address:

Listing 2. *Head and worker startup.*

```
# Head node (pc3-4500, 192.168.3.73)
ray start --head --port=6379 --dashboard-host=0.0.0.0

# Each worker node (repeat on Workers 1-7)
ray start --address='192.168.3.73:6379'
```

5.3 Cluster verification

After all workers joined, `ray status` from the head node confirmed 8 active nodes and the full pool of cluster resources:

Listing 3. *Live `ray status` output captured during the workshop.*

```
======== Autoscaler status: 2026-06-10 15:20:04.317026 ========
Node status
Active:
  8 nodes  (no failures)

Resources
  Total Usage:
  0.0/268.0  CPU
  0.0/8.0    GPU
  0B/336.69GiB  memory
  0B/144.29GiB  object_store_memory
```

5.4 Shared storage and MLflow

NFS shared storage at `/mnt/cluster_storage/` is mounted on all eight nodes and must be active before any training job is submitted. It holds three categories of data: MLflow run artifacts, training checkpoints (saved at configurable step intervals), and shared datasets.

An MLflow tracking server runs on the head node at port 5000. Every training script connects to it at startup so that metrics, parameters, and model artifacts are centralised and visible across the team.

6 Part 3 — Using the Cluster

6.1 Connecting from code

The entry point for any cluster job is `ray.init(address="auto")`. Combined with the `@ray.remote` decorator, this is the complete API surface for launching distributed work:

Listing 4. *Minimal cluster task: the pattern demonstrated in the workshop.*

```
import ray

ray.init(address="auto")

@ray.remote
def matmul():
    A = np.full((4096, 4096), 0.5, dtype=np.float64)
    B = np.full((4096, 4096), 0.5, dtype=np.float64)
    return socket.gethostname(), float(np.matmul(A, B).sum())

futures = [matmul.remote() for _ in range(300)]
results = ray.get(futures)
ray.shutdown()
```

6.2 Matrix multiplication — single machine vs. cluster

This benchmark was the first live demo of the session. Three hundred 4096×4096 matrix multiplications were dispatched, first serially on a single machine, then as `@ray.remote`

tasks across the cluster. Figure 4 shows the result.

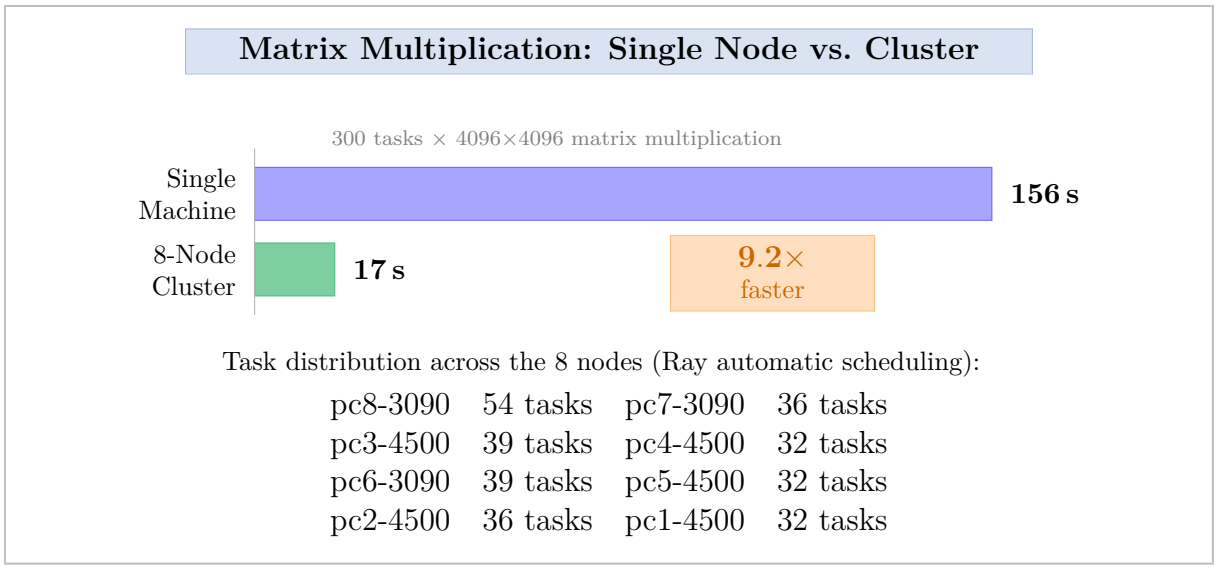


Figure 4. *Live benchmark result: 300 matrix multiplications on a single machine took 156 s; the same workload dispatched as Ray tasks across the 8-node cluster completed in 17 s — a 9.2× speedup. Task distribution was determined automatically by the Ray scheduler.*

6.3 Ray Train — distributed PyTorch

Nine training scripts of increasing complexity were presented, split into two categories: vision/ViT models (ResNet and ViT families, scripts 1–5) and large language models (GPT-2 and Llama families, scripts 6–9). Table 5 lists the full set.

Table 5. *Ray Train example scripts demonstrated during the workshop.*

#	Model	Training strategy	Category
1	ResNet-152	Single-GPU baseline	Vision / ViT
2	ResNet-152	DDP across all 8 GPUs	
3	ResNet-18	DDP + TensorBoard profiler	
4	ViT-Small	FSDP2	
5	ViT-H/14	FSDP2 (large model)	
6	GPT-2	FSDP2 on WikiText-2	LLMs
7	GPT-2	FSDP2 on TinyStories	
8	GPT-2	FSDP2 + MLflow tracking	
9	Llama 3.1-8B	FSDP2 across all 8 GPUs	

The Llama 3.1-8B FSDP training job (script 9) was the centrepiece of Part 3. All eight workers were confirmed active in the startup log:

Listing 5. *Ray Train log confirming 8-GPU FSDP worker group.*

```
[TrainController] Requesting resources: {'GPU': 1} * 8
[TrainController] Starting worker group of size 8:
  ip=192.168.3.78  world_rank=0  node_rank=0
```

```
  ip=192.168.3.76  world_rank=1  node_rank=1
  ip=192.168.3.77  world_rank=2  node_rank=2
  ip=192.168.3.73  world_rank=3  node_rank=3
  ip=192.168.3.72  world_rank=4  node_rank=4
  ip=192.168.3.74  world_rank=5  node_rank=5
  ip=192.168.3.71  world_rank=6  node_rank=6
  ip=192.168.3.75  world_rank=7  node_rank=7
[RayTrainWorker] FSDP2 sharding complete.
[MLflow] Run: http://192.168.3.73:5000/#/experiments/5/runs/47
   b5d38fa77740c9b24...
```

The Ray Dashboard showed the job completing with status **SUCCEEDED** after **2 h 49 min**, with all 10 000 Ray tasks finishing without failures.

6.4 vLLM with Ray — multi-node LLM inference

Llama 3.1-8B was deployed as a live inference server spread across all eight GPU nodes using vLLM's pipeline-parallelism with Ray as the distributed executor. Figure 5 shows the serving pipeline.

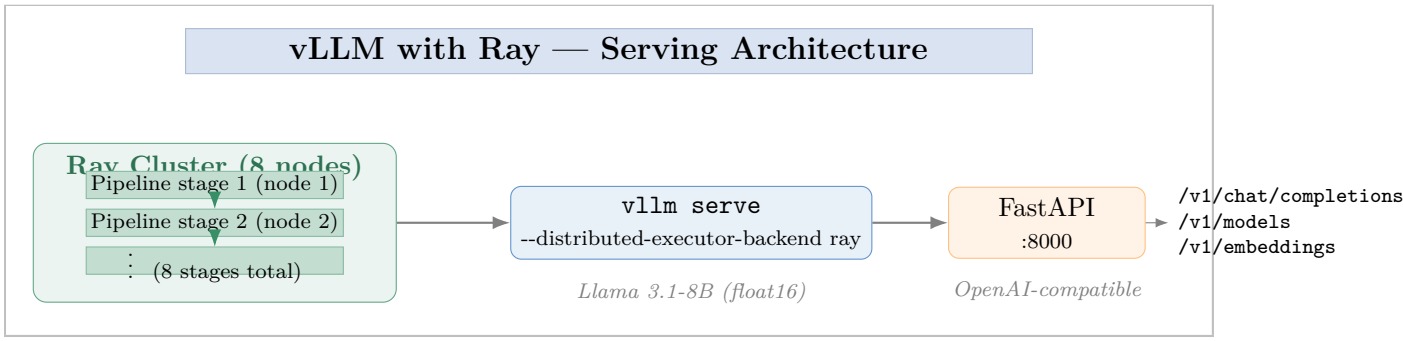


Figure 5. *vLLM pipeline-parallel serving architecture demonstrated in the workshop. The model is sharded across 8 pipeline stages (one per GPU node); the Ray distributed executor coordinates token passing. The FastAPI server at port 8000 exposes an OpenAI-compatible HTTP API.*

The launch command issued on the head node:

Listing 6. *vLLM multi-node launch command.*

```
RAY_ADDRESS=192.168.3.73:6379 VLLM_PLUGINS="" \
vllm serve ~/projects/vllm-deployment/vllm/models/3.1-8b-instruct \
  --dtype float16 \
  --tensor-parallel-size 1 \
  --pipeline-parallel-size 7 \
  --distributed-executor-backend ray \
  --gpu-memory-utilization 0.4 \
  --max-model-len 1024 \
  --enforce-eager \
  --host 0.0.0.0 --port 8000
```

All eight nodes appeared as **ALIVE** in the Ray Dashboard with active GRAM usage once the model had loaded. The `/v1/models` endpoint confirmed `3.1-8b-instruct` as the active serving model.

6.5 MLflow experiment tracking

Every training run was tracked automatically. Figure 6 summarises the MLflow pipeline used throughout Part 3.

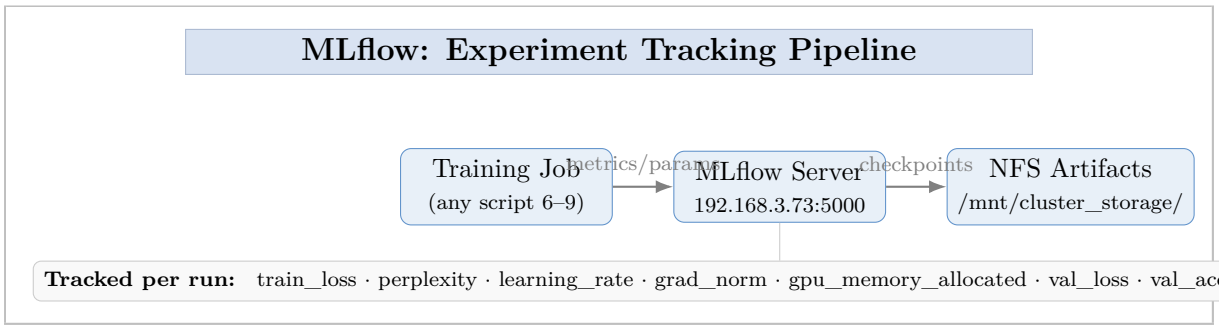


Figure 6. *MLflow experiment tracking pipeline used throughout the workshop. Every training script logs metrics and parameters to the head-node server; model checkpoints are written to NFS shared storage at configurable step intervals.*

The live MLflow UI showed the Llama 3.1-8B run (named `llama31_8b_fsdp`) with 12 tracked metrics and 30 recorded parameters (model path, `num_parameters`: 8 030 000 000, `num_layers`: 32, `hidden_size`: 4096, etc.). Checkpoints were saved at 500-step intervals up to step 7500.

7 Part 4 — Hands-On Examples

7.1 Examples status

Table 6 records which examples were completed during the workshop and which are planned for future sessions, exactly as shown on the final status slide of the presentation.

Table 6. *Workshop examples — completion status.*

#	Folder	Library	Status
1	`1-ray-core/`	Ray Core	Completed
2	`2-ray-training/`	Ray Train	Completed
3	`3-RL-lib/`	Ray RLlib	Planned
4	`4-ray-serve(LLM)/`	Ray Serve	Planned
5	`5-ray-tune/`	Ray Tune	Planned
6	`6.vllm/`	vLLM + Ray	Completed

7.2 Completed examples

Example 1 — Ray Core. Two side-by-side scripts demonstrated the `@ray.remote` pattern: square-root computations and matrix multiplications (the latter producing the 9.2× speedup result above). Participants could observe the dashboard distributing tasks in real time.

Example 2 — Ray Train. The training curriculum ran from a single-GPU ResNet-152 baseline through DDP, FSDP2 on ViT-H/14, and finally FSDP2 on Llama 3.1-8B across

all eight cluster GPUs. The Llama job completed successfully in 2 h 49 min. Three key implementation patterns were highlighted: DDP via `prepare_model()`/`prepare_data_loader()`; FSDP2 via `torch.distributed.fsdp.fully_shard()` with a device mesh and "meta"-device model loading to avoid CPU OOM; and Ray Train V2 enabled with `os.environ["RAY_TRAIN_V2_E`

Example 6 — vLLM on 8 GPUs. The end-to-end serving workflow was demonstrated: start cluster, launch vLLM with Ray backend, verify GPU allocation in the Dashboard, and query the OpenAI-compatible endpoint. The `/v1/chat/completions` and `/v1/models` endpoints both responded correctly.

7.3 Planned examples

Three examples are scheduled for subsequent sessions: *Ray RLlib* (PPO reinforcement learning with automatic rollout workers), *Ray Serve* (autoscaling model serving with rolling updates), and *Ray Tune* (distributed hyperparameter search using the Optuna backend).

8 Key Technical Points

Several non-obvious implementation details were covered in the workshop and are worth recording here for reference.

1. **Always declare resources on `@ray.remote`.** Omitting `num_gpus` and `num_cpus` causes Ray to default to 1 CPU and 0 GPUs per task, which can monopolise a single node or leave all GPUs idle.

2. **Use `ray.put()` for large shared data.** Passing a 10 GB dataset directly into many task calls creates duplicate copies in standard RAM. `ray.put()` stores the object once in the zero-copy shared object store; workers receive a lightweight `ObjectRef`.

3. **Submit via the Job CLI, not direct execution.** `python my_script.py` runs the driver locally and logs are only visible in the terminal. `ray job submit ...` routes everything through the Dashboard and makes `print()` output, errors, and task graphs visible live in the browser.

4. **`GLOO_SOCKET_IFNAME` must be in the shell at `ray start` time.** A running Ray worker does not re-read `.bashrc`. Without this variable, Gloo auto-detects Docker bridge interfaces (172.x.x.x range) and the `connectFullMesh` handshake fails. After any change, stop and restart the worker so it inherits the updated environment.

5. **FSDP large-model initialisation.** Loading a model to CPU first and then sharding it with FSDP2 exhausts RAM for 8B+ parameter models. The correct pattern is to create the model on the `"meta"` device, apply `fully_shard()`, then call `.to_empty(device=device)` to allocate real memory only after sharding is complete.

6. **Graceful shutdown.** Always call `ray.shutdown()` at the end of driver scripts to release GPU and CPU reservations back to the cluster pool.

9 Conclusion

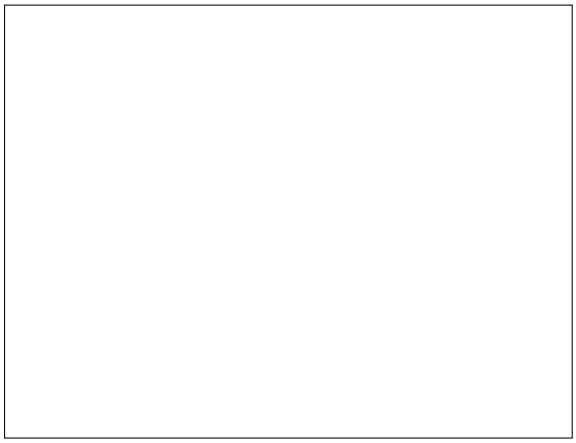
The workshop successfully demonstrated the complete Ray cluster workflow on live hardware. Three outcomes are immediately reproducible by participants:

- A bare-metal eight-node Ray cluster can be assembled in under fifteen minutes using the Manual CLI method and verified with `ray status`.
- Distributed PyTorch training — up to and including an 8-billion-parameter FSDP-sharded language model — can be submitted as a single `ray job submit` command and tracked end-to-end in MLflow and the Ray Dashboard.
- A multi-node LLM inference server (Llama 3.1-8B, pipeline-parallel across 8 GPUs) can be launched with a single `vllm serve` command using `--distributed-executor-backend ray`, and is immediately accessible through an OpenAI-compatible HTTP API.

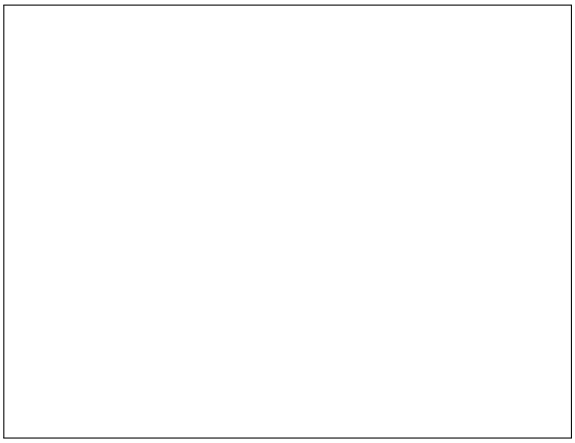
The three planned examples (Ray RLlib, Ray Serve, Ray Tune) will be covered in a follow-up session, completing the full `examples/` library in the workshop repository.

Workshop Photographs

The following pages contain photographs taken during the workshop session on 19 June 2026.

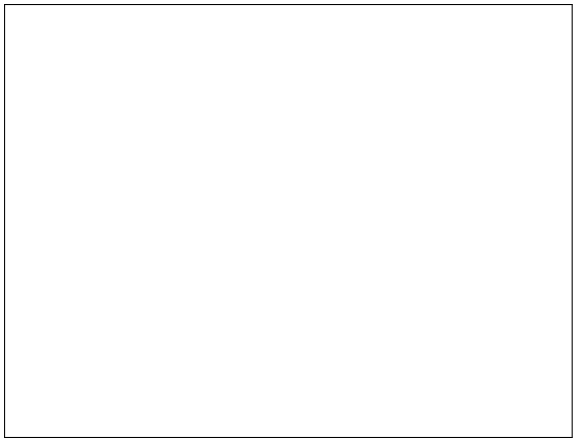


(a) *Participants during the foundations section.*

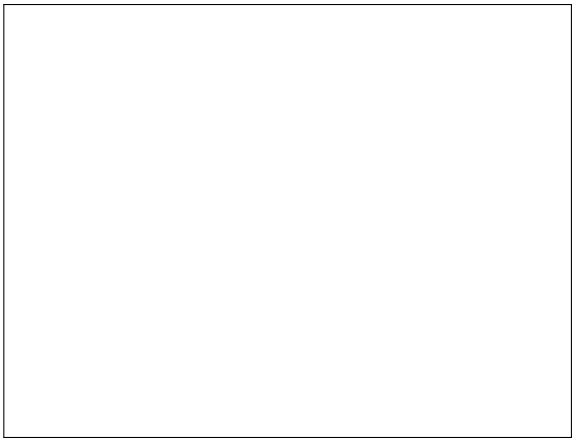


(b) *Physical cluster hardware setup (8 nodes).*

Figure 7. *Workshop session — Part 1: Foundations.*



(a) *Ray Dashboard showing all 8 nodes ALIVE.*



(b) *Matrix multiplication benchmark live result.*

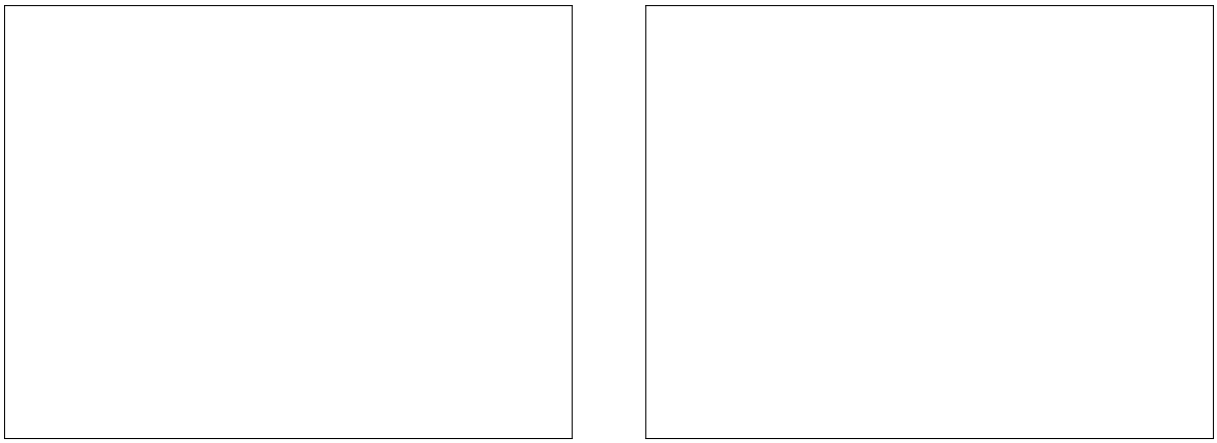
Figure 8. *Workshop session — Part 2 & 3: Deployment and using the cluster.*

Figure 9. *Llama 3.1-8B FSDP training job — SUCCEEDED in 2 h 49 min.*

(a) *vLLM + Ray: 8 GPUs serving Llama 3.1-8B.*

(b) *MLflow dashboard showing run metrics.*

Figure 10. *Workshop session — Part 4: Hands-on vLLM demonstration.*



(a) *Q&A and discussion.* **(b)** *Group photograph.*

Figure 11. *Workshop close.*

To insert real photographs: replace each `\fbox{...}` *placeholder with* `\includegraphics[width=\linew` *and place the image files in a* `photos/` *folder next to this* `.tex` *file.*